Certified Block Overlap in Subset Sum

Small local dictionaries, bounded search, paid discovery

Jonathan R. Landers November 2026

Abstract. With n input weights, subset sum can admit a short exact search even when all 2^n sums are distinct. We divide sorted positive weights into small blocks and list each block's distinct sums. An interval test bounds the choices that survive in each block; their product bounds the search tree. A dynamic program minimizes this product over contiguous partitions with at most b items per block. Writing the optimum as K_b and binary input length as L, finding the partition and solving the instance take

$$\operatorname{poly}(L)(2^b + K_b) \quad \text{bit operations},$$

where $\operatorname{poly}(L)$ is a polynomial factor in L. We prove the bound, give a robust family with $K_3 = 1$, and verify the implementation exhaustively on small inputs.

1 Why local separation helps

Subset sum asks whether some selection of positive integer weights totals a specified integer target. Merging repeated sums is one shortcut. We study another: *separation* between local sums can rule out most choices, even when all global sums differ.

Consider the blocks $(2, 3, 4)$ and $(20, 30, 40)$. The lower block totals 9. To reach 37, the higher block must contribute between 28 and 37. Only 30 fits, leaving 7, which the lower block can reach. For target 38, the same choice leaves 8, which it cannot reach. The interval test rules out whole groups, but a surviving choice still needs an exact check. We bound this search for every target and include the cost of finding the partition.

The meet-in-the-middle algorithm of Horowitz and Sahni [1] uses $O^*(2^{n/2})$ time; Schroeppel and Shamir [2] retain that time with $O^*(2^{n/4})$ space. Here O^* suppresses factors polynomial in encoded input length. Ordered-sum constructions [3] already show that distinct sums can coexist with efficient decision. Li and Ye [4] improve on the classical $1/2$ exponent using compatibility tests on partial representations. We instead seek an easily checked condition that makes a particular input easy.

The input consists of positive integer weights and an integer target:

$$a = (a_1, \ldots, a_n) \in \mathbb{Z}_{>0}^n, \qquad t \in \mathbb{Z}, \qquad [n] = \{1, \ldots, n\}.$$

The required output is an index set $U \subseteq [n]$ satisfying

$$\sum_{i \in U} a_i = t,$$

or a correct NO answer. Such a set U is a *witness*. Equal values remain distinct indexed items; the empty selection is allowed. Let L be the binary encoding length of (a, t), including delimiters. Then $n = O(L)$, and sums and witness masks (one bit per original item) have $O(L)$ bits. All logarithms are base two. The notation $\operatorname{poly}(L)$ hides a uniform polynomial, with no extra exponential parameter. Numeric magnitude is different: for $t \geq 0$, Bringmann's randomized $\widetilde{O}(n + t)$ algorithm [5] is near-linear pseudopolynomial time, not polynomial time in the binary length of t. The tilde suppresses logarithmic factors.

2 Local sums and the interval test

Sort the items as $w_1 \leq \cdots \leq w_n$, breaking ties by original index and retaining that index map. Choose an integer block-size limit $1 \leq b \leq n$. Partition the indices into m nonempty blocks using integer cuts

$$0 = v_0 < v_1 < \cdots < v_m = n, \qquad v_j - v_{j-1} \leq b.$$

The jth block is the index set

$$B_j = \{v_{j-1} + 1, \ldots, v_j\}.$$

Define its distinct-sum set S_j and the total weight P_j of all lower blocks:

$$S_j = \left\{ \sum_{i \in U} w_i : U \subseteq B_j \right\}, \qquad P_j = \sum_{i=1}^{v_{j-1}} w_i. \tag{1}$$

Store the sorted distinct values in S_j, keeping one original-index witness per value. If higher blocks leave residual target r, a completion using $s \in S_j$ needs a lower contribution satisfying

$$r - s \in [0, P_j].$$

A set of attainable sums is also called a *support*. Bringmann, Fischer, and Nakos [6] compute the support $\mathcal{S}_{\le t}$ of all attainable sums up to a nonnegative target t in randomized time

$$\widetilde{O}\big(|\mathcal{S}_{\le t}|\sqrt{n}\big).$$

We build only small local supports, avoiding the full set in the family below.

Lemma 1 (Sound exclusion and merging)**.** *Only local sums satisfying*

$$s \in S_j \cap [r - P_j, r]$$

can occur in a completion. Keeping one witness per distinct block sum preserves existence and recovery of one full witness.

Proof. Every lower selection has sum u with $0 \le u \le P_j$. Therefore

$$s + u = r \quad \Longrightarrow \quad r - P_j \le s \le r.$$

Conversely, two selections within B_j with equal sum leave the same residual and the same unprocessed blocks. Substituting one for the other preserves a solution and cannot reuse an index, because blocks are disjoint. □

The interval $[s, s + P_j]$ contains every residual target that could use local sum s. Write $\mathcal{B} = (B_1, \dots, B_m)$ for the partition. Its maximum local overlaps κ_j and their product $K(\mathcal{B})$ are

$$\kappa_j = \max_{z \in \mathbb{Z}} |S_j \cap [z - P_j, z]|, \qquad K(\mathcal{B}) = \prod_{j=1}^{m} \kappa_j, \tag{2}$$

$$1 \le \kappa_j \le |S_j| \le 2^b.$$

When the partition is fixed, we abbreviate $K(\mathcal{B})$ as K. Intervals are closed: sums exactly P_j apart can both qualify. Figure 1 makes this endpoint issue visible.

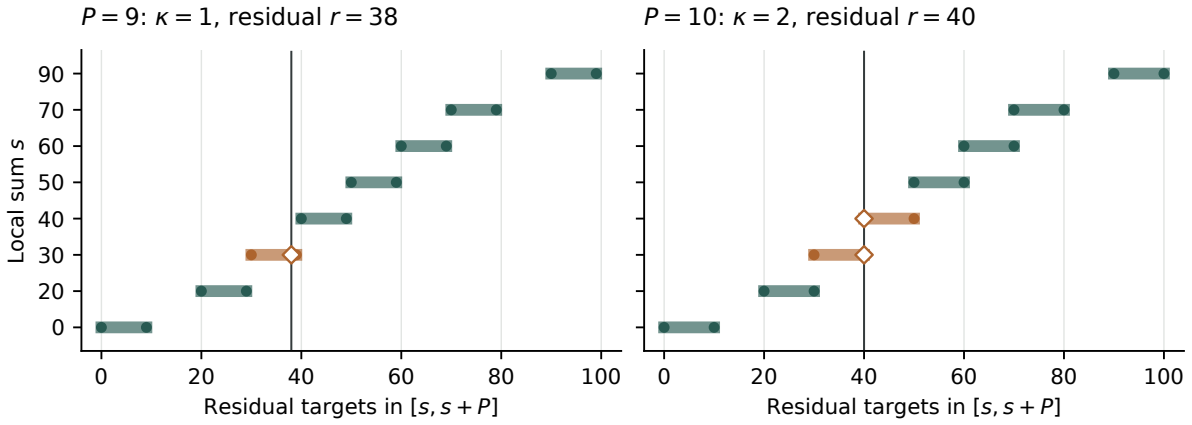


Figure 1: **Overlap is a branching certificate.** Segments show $[s, s + P]$ for the local sums of $(20, 30, 40)$; P and κ denote this block's lower total and maximum overlap. Orange marks the choices that contain the indicated residual. For $P = 9$, at most one choice survives. For $P = 10$, residual 40 lies at the shared endpoints for $s = 30$ and $s = 40$, so $\kappa = 2$. The original example uses $P = 9$; the right panel varies the lower total.

3 Bounding the search tree

Search blocks in descending order; depth counts processed blocks, starting at zero. At block j, use two binary searches to locate

$$S_j \cap [r - P_j, r],$$

then recurse on each qualifying sum s with residual $r - s$. At depth m, accept precisely residual zero. Reject $t \notin [0, \sum_i w_i]$ immediately.

Theorem 2 (Fixed-partition search)**.** *The algorithm is exact. Let $N(\mathcal{B}, t)$ count the nodes in its complete search tree for target t. Then*

$$N(\mathcal{B}, t) \le 1 + \sum_{\ell=1}^{m} \prod_{h=m-\ell+1}^{m} \kappa_h \le (m+1)K(\mathcal{B}). \tag{3}$$

There are at most $K(\mathcal{B})$ complete paths, each choosing one sum per block. After construction of the local dictionaries, its bit cost is at most $\mathrm{poly}(L)K(\mathcal{B})$*. An early YES may stop sooner.*

Proof. Induct on the number of remaining blocks. With no blocks, precisely residual zero is attainable. Lemma 1 discards no solution and justifies the stored representative for every surviving local sum. Applying the induction hypothesis to these children proves both YES and NO correctness.

At depth ℓ, each preceding node has at most $\kappa_{m-\ell+1}$ children, independently of its residual. Multiplying these deterministic upper bounds and summing levels gives the first inequality in (3). Since every $\kappa_h \ge 1$, every partial product is at most $K(\mathcal{B})$, proving the second. Binary searches take

$$O\big(\log(|S_j| + 1)\big) = O(b + 1)$$

comparisons per internal node. Charge each generated child once; sums, residuals, and masks have $O(L)$ bits. Witness assembly and its final weighted-sum check are polynomial. All these charges fit the stated bit bound. □

The bound includes failed branches and NO instances; it uses no independence assumption. Merging equal-sum witnesses preserves existence, not solution counts.

4 Finding and building the partition

A small product helps only if its partition is cheap to find. For a contiguous block, the lower total depends only on its starting position. It stays fixed as earlier cuts change, giving a dynamic program.

Define the prefix totals by

$$H_v = \sum_{i=1}^{v} w_i, \qquad H_0 = 0.$$

For $1 \le u \le v \le n$ and $v - u + 1 \le b$, let $S_{u,v}$ be the distinct sums of $w_u, \dots, w_v$ and define its maximum overlap as

$$\kappa(u, v) = \max_{z \in \mathbb{Z}} |S_{u,v} \cap [z - H_{u-1}, z]|.$$

Let $D(v)$ be the minimum overlap product over permitted partitions of the first v items. The exact integer recurrence

$$\begin{aligned} D(0) = 1, \qquad K_b = D(n), \\ D(v) = \min_{\max(1, v-b+1) \le u \le v} D(u-1)\kappa(u, v) \end{aligned} \tag{4}$$

stores a minimizing predecessor for each v.

Theorem 3 (Optimal contiguous discovery and total cost)**.** *The recurrence finds*

$$K_b = \min_{\mathcal{B}} K(\mathcal{B}),$$

where the minimum ranges over all contiguous sorted partitions whose blocks contain at most b items. For the returned partition of m blocks, let $T(a,t;b)$ be the total bit-operation count and $\mathrm{Space}(a,t;b)$ *the peak workspace in bits. Including sorting, construction, optimization, reconstruction, and search,*

$$\boxed{\begin{aligned} T(a,t;b) &\le \mathrm{poly}(L)(2^b + K_b), \\ \mathrm{Space}(a,t;b) &\le \mathrm{poly}(L)2^b. \end{aligned}} \tag{5}$$

Optimality refers to this partition family and product bound.

Proof. Every partition of the first v items has a last block $[u,v]$. Its final factor $\kappa(u,v)$ uses H_{u-1}, regardless of earlier cuts. Replacing the preceding partition by an optimum cannot increase the product. Induction proves (4) and predecessor reconstruction.

There are at most nb candidate blocks. For a block of length ℓ, start with $(0,\varnothing)$ and repeatedly merge the current sorted sum list with its translate by the next weight, coalescing equal sums and retaining a witness. List sizes are at most 2^h after h items; the total merge work is

$$O\left(\sum_{h=1}^{\ell} 2^h\right) = O(2^\ell)$$

arithmetic/comparison operations.

List a candidate support as $s_1 < \cdots < s_q$, with $q = |S_{u,v}|$. A sliding window moves its left endpoint forward until

$$s_{\mathrm{right}} - s_{\mathrm{left}} \le H_{u-1},$$

where *left* and *right* index the window endpoints. The largest window equals $\kappa(u,v)$: any nonempty eligible interval can move its right endpoint to its largest contained support value without losing a contained value. Each endpoint moves at most q times, so scoring takes $O(q)$ comparisons.

Thus discovery costs

$$O\big(n\log(n+1) + nb2^b\big) \quad \text{operations.}$$

Discard a candidate list after scoring, store its score, and rebuild only the chosen block lists, adding $O(n2^b)$ operations. Each DP candidate product is at most 2^v, so it uses at most $n+1$ bits. All other represented integers and witnesses also have polynomial bit length. Theorem 2 then gives (5). More explicitly, schoolbook integer arithmetic permits the conservative bound

$$O\big(L^2\left[n\log(n+1) + nb2^b + (m+1)(b+1)K_b\right]\big).$$

Store $O(nb)$ scores, predecessors, one temporary candidate list, and the chosen lists, which have at most $m2^b$ records. Depth-first search keeps one path and pending siblings: at most $1 + m(2^b - 1)$ pending nodes. Each record uses polynomially many bits, proving the space bound. □

Reference algorithm.

1. Sort with original indices; compute prefix totals H_v.
2. Construct and score every candidate block of length at most b.
3. Evaluate (4) using exact integer products; reconstruct its cuts and rebuild their witnessed dictionaries.
4. Search downward using Lemma 1; verify any recovered witness against the original input. Exhausting every surviving branch certifies NO.

For $n = 0$, take the empty partition and $K_b = 1$: only $t = 0$ is reachable.

Corollary 4 (An efficiently recognizable tractable class)**.** *Fix integers $c \geq 1$ and $d \geq 0$. For nonempty inputs choose*

$$b = \min\bigl\{n, \max\{1, \lfloor c \log_2(L+1) \rfloor\}\bigr\}.$$

The class of inputs satisfying

$$K_b \leq (L+1)^d$$

is recognizable and solvable in polynomial bit time, without a supplied partition.

Proof. Discovery is polynomial because 2^b is polynomial in L; $\lfloor c \log_2(L+1) \rfloor$ is computable as the bit length of $(L+1)^c$ minus one. Compare the computed integer K_b with the threshold and apply Theorem 3 on accepted inputs. □

Other algorithms exploit different structure. Bringmann and Wellnitz [7] study dense inputs, with algorithms and conditional lower bounds. For a nonempty integer set A, its *sumset* is

$$A + A = \{x + y : x, y \in A\}.$$

Randolph and Węgrzycki [8] use its additive doubling $|A + A|/|A|$. Our κ_j instead counts local choices compatible with a lower total.

Increasing b cannot increase K_b: every earlier partition remains available. But larger blocks can cost more to build. At $b = n$, a single block gives $K_b = 1$, yet its support may contain 2^n records. This is why the bound charges 2^b separately. Stored dictionaries and generated sums also drive time–space methods such as OpenAI's low-space preprint [9], which uses a different randomized construction and word model.

5 Many distinct sums, one search path

For an integer $m \geq 1$, take m sorted triples. For $j = 0, \ldots, m-1$, assign the following weights to block B_{j+1}:

$$\begin{aligned} \{w_i : i \in B_{j+1}\} &= 10^j\{2, 3, 4\}, \qquad n = 3m, \\ S_{j+1} &= 10^j\{0, 2, 3, 4, 5, 6, 7, 9\}. \end{aligned} \tag{6}$$

Multiplication by 10^j scales each element of the displayed sets. The total below block $j+1$ is

$$P_{j+1} = 9 \sum_{h=0}^{j-1} 10^h = 10^j - 1.$$

Every gap between its consecutive support values is at least 10^j, strictly exceeding P_{j+1}. Hence

$$\kappa_j = 1 \quad \text{for every block,} \qquad K_3 = 1.$$

The search visits at most $m + 1$ nodes for any target.

Each triple's eight selections produce eight different decimal digits, and no digit carries into the next block. Consequently,

$$\text{distinct global sums} = 8^m = 2^n, \qquad \text{retained local records} = 8m.$$

Because $b = 3$, discovery is polynomial, too: the algorithm never builds the global support.

A sequence is *superincreasing* if each weight exceeds the sum of all earlier weights. This family fails that condition:

$$4 \cdot 10^j < 2 \cdot 10^j + 3 \cdot 10^j.$$

The maximum-core greedy scan used in the core-and-exceptions approach [10] keeps the first two items of each triple and rejects the third. Indeed, before each triple its retained total is

$$\frac{5(10^j - 1)}{9} < 10^j;$$

the first two items exceed the preceding retained total, while the third does not. That route leaves m exceptions and its enumeration bound is exponential. This compares sufficient bounds, not lower bounds against other algorithms.

Proposition 5 (Irregular scales and perturbations)**.** *For blocks $j = 1, \ldots, m$, replace the weights in B_j by*

$$\{2M_j + e_{j,2}, 3M_j + e_{j,3}, 4M_j + e_{j,4}\},$$

where $M_j, E_j \in \mathbb{Z}_{\geq 0}$ are a scale and an error bound. The integer errors $e_{j,h}$ satisfy $|e_{j,h}| \leq E_j$ for $h \in \{2, 3, 4\}$; P_j is the actual total of preceding triples. If

$$M_j - 3E_j > P_j \qquad \text{for every } j, \tag{7}$$

the triples form a contiguous sorted partition with $\kappa_j = 1$, all 2^{3m} global sums are distinct, and $b = 3$ gives polynomial bit time.

Proof. An unperturbed difference between two distinct local subset sums has absolute value at least M_j. Each of the three errors appears in their difference with coefficient in $\{-1, 0, 1\}$, changing it by at most $3E_j$. Thus every local difference remains larger than P_j. Condition (7) also ensures positive weights, increasing order within each triple, and a new smallest weight exceeding the earlier total. The triples are therefore permitted sorted blocks with unit overlap. For two different global selections, their highest differing block contributes a difference larger than the total of all lower blocks; cancellation is impossible. Apply Theorem 3. □

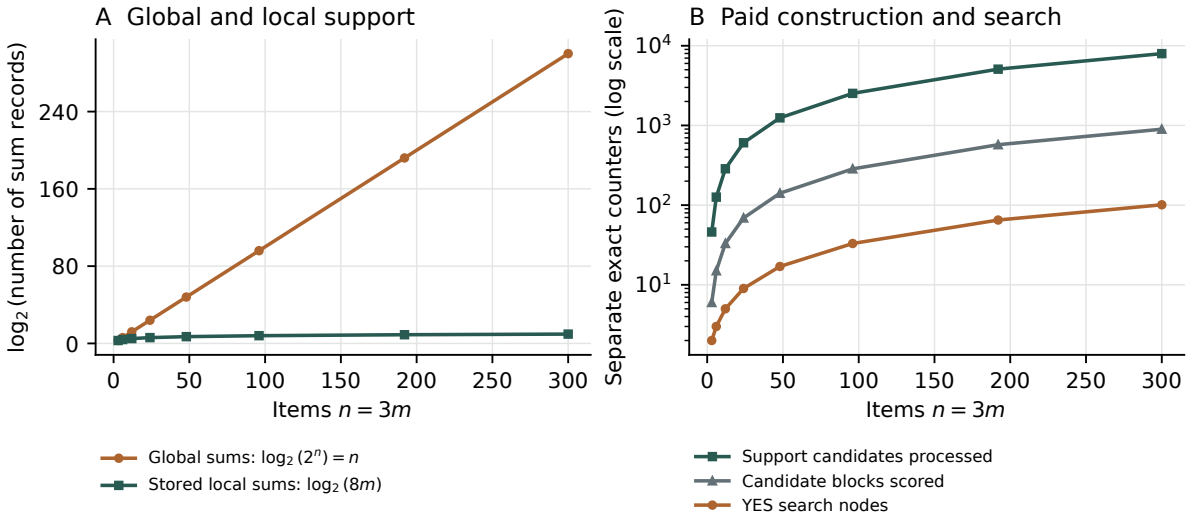


Figure 2: **The separated family, with discovery visible.** With $b = 3$ and $m \in \{1, 2, 4, 8, 16, 32, 64, 100\}$, panel A compares global and stored local support sizes on a base-two logarithmic scale. Panel B separates construction counters from search nodes for the all-7 YES target. Support candidates include discovery and rebuilding. These counters are neither timings nor uniform bit operations; they are not added together.

6 Verification and scope

The standard-library solver uses exact integers and returns original indices. An independent checker enumerates subsets and all admissible cut sets on small inputs. The saved record reports:

Check or constructed-example counter	Count
All sorted multisets of 0–6 items from $\{1, \ldots, 5\}$	462
Partition-optimality checks, $b \in \{1, 2, 3\}$	1,386
Additional seeded random inputs, at most 12 items	120
Decision/witness comparisons against exhaustive subsets	37,055
Candidate blocks scored, $n = 300$, $b = 3$	897
Support candidates, including reconstruction	7,966
Search nodes: all-7 YES target / leading-8 NO target	101 / 1

Each tested target checks the decision, the original-index witness when present, and (3)'s $(m+1)K$ envelope. Boundary checks cover closed endpoints, duplicate weights, empty inputs, and invalid weights. The seed for the random inputs is 20261104. For the 300-item example, the NO target is $8 \cdot 10^{100-1}$; the global count 2^{300} follows from the proof, not exhaustive enumeration. Global support distinctness is also checked directly on the first one, two, and three triples. These are correctness checks, not timing benchmarks.

The source directory contains `solver.py`, `verify.py`, and `validation.json`, which records their SHA-256 hashes. From the repository root, run `python notes/01-certified-coverage/block-overlap/verify.py` to repeat the checks, or run the same directory's `build_note.py` to rebuild the paper. The builder checks hashes and citations, regenerates plots and numerical macros, records `paper_evidence.json`, and checks layout without a page limit. It preserves the original verification record.

Relation to distribution shaping. For a fixed partition,

$$\log_2 K = \sum_{j=1}^{m} \log_2 \kappa_j.$$

This sum could motivate a statistical model across random inputs, but does not itself imply a lognormal law. That requires assumptions on the input distribution and the terms' dependence. Changing an axis leaves the computation unchanged; the saving comes from bounded overlap.

Prior work and limits. Singleton blocks $B_j = \{j\}$ with $w_j > P_j$ recover the classical superincreasing case. Earlier ordered-sum constructions already combine distinct subset sums with efficient knapsack decision [3]; that report's abstract establishes precedent, not equivalence with this overlap bound. We give a costed proof and checked implementation. Novelty assessment, comparative timings, and partitions beyond contiguous sorted blocks remain open. Small $2^b + K_b$ guarantees an efficient route; large K_b does not prove hardness.

References

[1] E. Horowitz and S. Sahni. *Computing Partitions with Applications to the Knapsack Problem.* Journal of the ACM 21(2), 277–292, 1974. doi:10.1145/321812.321823.

[2] R. Schroeppel and A. Shamir. *A $T = O(2^{n/2})$, $S = O(2^{n/4})$ Algorithm for Certain NP-Complete Problems.* SIAM Journal on Computing 10(3), 456–464, 1981. doi:10.1137/0210033.

[3] M. D. Atkinson, A. Negro, and N. Santoro. *Sums of Lexicographically Ordered Sets.* Carleton University Technical Report TR-100, May 1987. Institutional abstract.

[4] B. Li and L. Ye. *Subset Sum via Partial Match.* Preprint, version 2, November 2, 2026. arXiv:2610.10446v2.

[5] K. Bringmann. *A Near-Linear Pseudopolynomial Time Algorithm for Subset Sum.* SODA, 2017. arXiv:1610.04712v2.

[6] K. Bringmann, N. Fischer, and V. Nakos. *Beating Bellman's Algorithm for Subset Sum.* SODA, 2025. arXiv:2410.21942v1.

[7] K. Bringmann and P. Wellnitz. *On Near-Linear-Time Algorithms for Dense Subset Sum.* SODA, 2021. arXiv:2010.09096.

[8] T. Randolph and K. Węgrzycki. *Parameterized Algorithms on Integer Sets with Small Doubling: Integer Programming, Subset Sum and k-SUM.* ESA, 96:1–96:19, 2024. doi:10.4230/LIPIcs.ESA.2024.96.

[9] OpenAI. *A Low-Space Algorithm for Worst-Case Subset Sum.* Preprint dated September 26, 2026; fixed repository version.

[10] J. R. Landers. *How Far from Easy? Structural Distance in Subset Sum.* Working draft, 2026. Repository Note V, source and evidence.